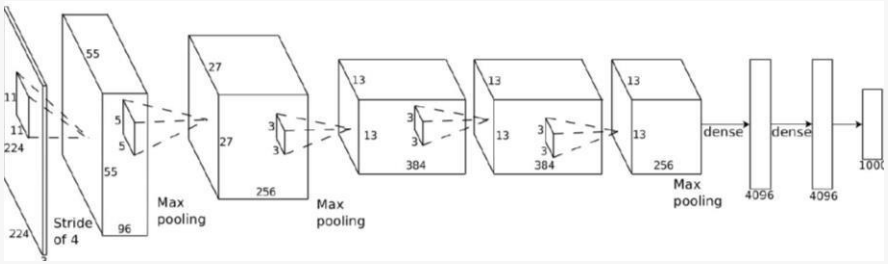
Reinforcement Learning – Other Topics 2

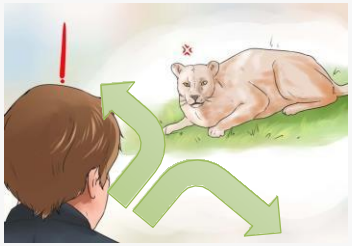
Francisco Giral

Topics..

- Tips and Tricks for Practical RL
- **Advanced Topics:**
  - Model-based Reinforcement Learning
  - Imitation Learning
  - RL in Language Models
  - Multi-Agent RL

Terminology & notation





$\mathbf{o}_t$ $\pi_\theta(\mathbf{a}_t|\mathbf{o}_t)$ $\mathbf{a}_t$

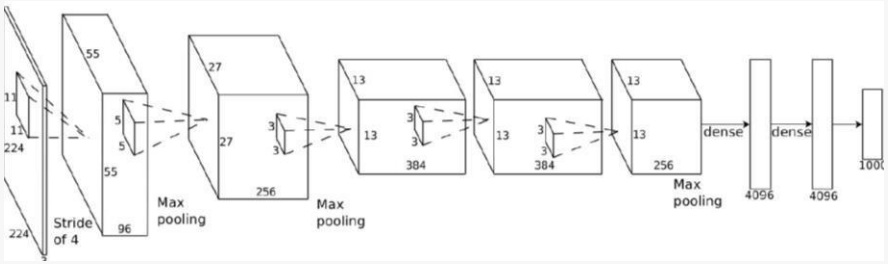
$\mathbf{s}_t$ – state
$\mathbf{o}_t$ – observation
$\mathbf{a}_t$ – action

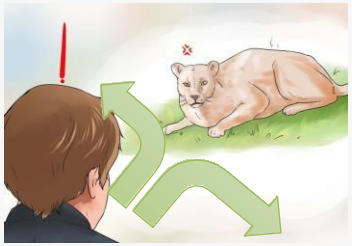
$\pi_\theta(\mathbf{a}_t|\mathbf{o}_t)$ – policy
$\pi_\theta(\mathbf{a}_t|\mathbf{s}_t)$ – policy (fully observed)

$\mathbf{o}_t$ – observation

$\mathbf{s}_t$ – state

Terminology & notation

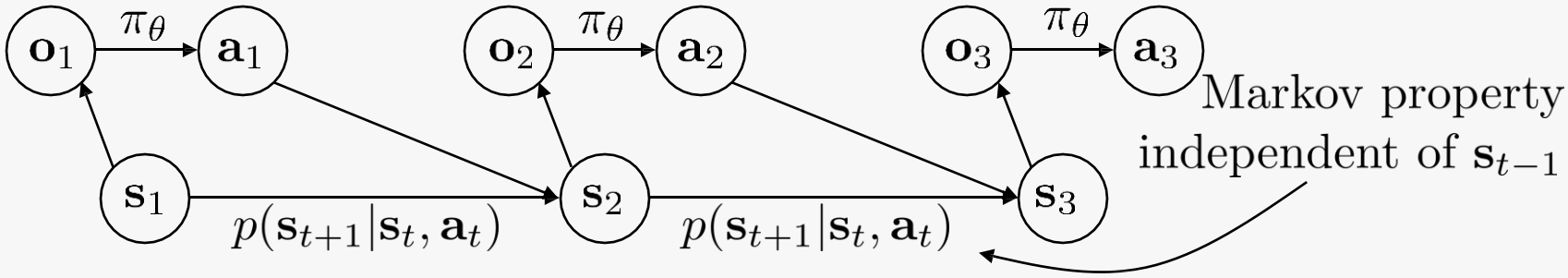




$\mathbf{o}_t$ $\pi_\theta(\mathbf{a}_t|\mathbf{o}_t)$ $\mathbf{a}_t$

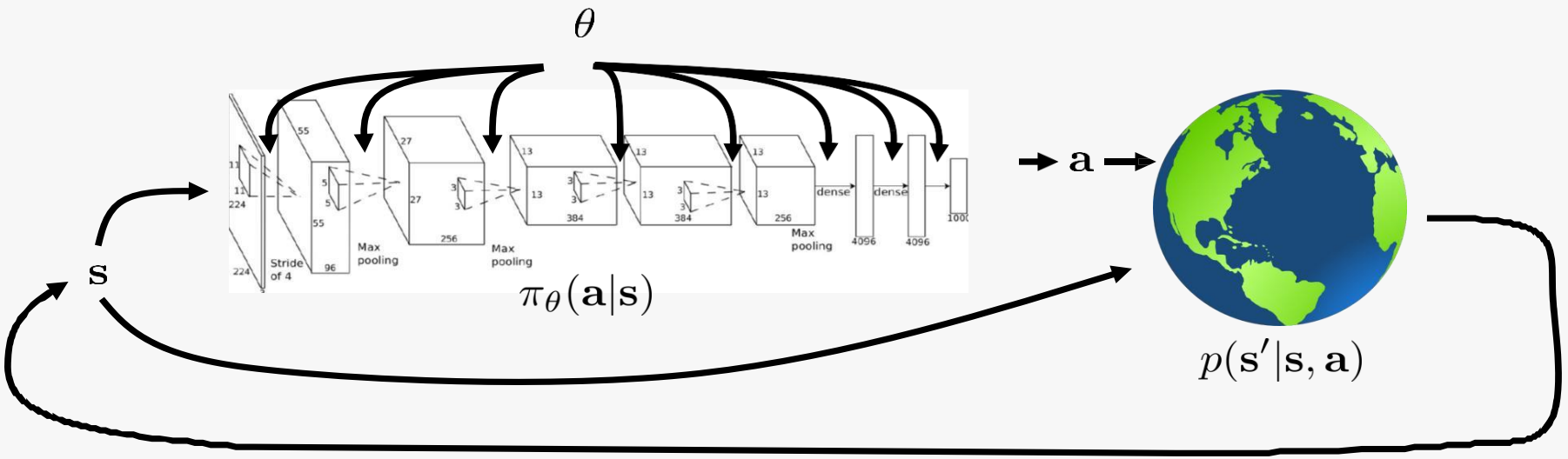
$\mathbf{s}_t$ – state
$\mathbf{o}_t$ – observation
$\mathbf{a}_t$ – action

$\pi_\theta(\mathbf{a}_t|\mathbf{o}_t)$ – policy
$\pi_\theta(\mathbf{a}_t|\mathbf{s}_t)$ – policy (fully observed)



Model-Based Reinforcement Learning

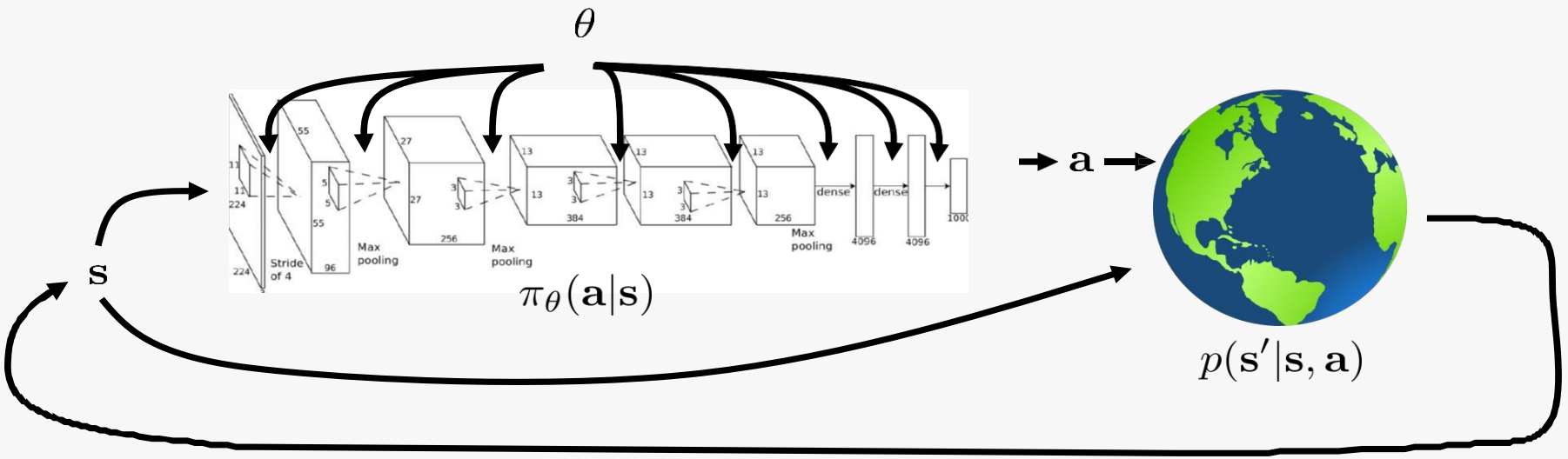
Recap: the reinforcement learning objective



$$\underbrace{p_\theta(\mathbf{s}_1, \mathbf{a}_1, \ldots, \mathbf{s}_T, \mathbf{a}_T)}_{\pi_\theta(\tau)} = p(\mathbf{s}_1) \prod_{t=1}^{T} \pi_\theta(\mathbf{a}_t|\mathbf{s}_t) p(\mathbf{s}_{t+1}|\mathbf{s}_t, \mathbf{a}_t)$$

$$\theta^\star = \arg\max_\theta E_{\tau \sim p_\theta(\tau)} \left[\sum_t r(\mathbf{s}_t, \mathbf{a}_t) \right]$$

Recap: the reinforcement learning objective



$$\underbrace{p_\theta(\mathbf{s}_1, \mathbf{a}_1, \ldots, \mathbf{s}_T, \mathbf{a}_T)}_{\pi_\theta(\tau)} = p(\mathbf{s}_1) \prod_{t=1}^{T} \pi_\theta(\mathbf{a}_t|\mathbf{s}_t) p(\mathbf{s}_{t+1}|\mathbf{s}_t, \mathbf{a}_t)$$

assume this is unknown

don't even attempt to learn it

$$\theta^\star = \arg\max_\theta E_{\tau \sim p_\theta(\tau)} \left[\sum_t r(\mathbf{s}_t, \mathbf{a}_t) \right]$$

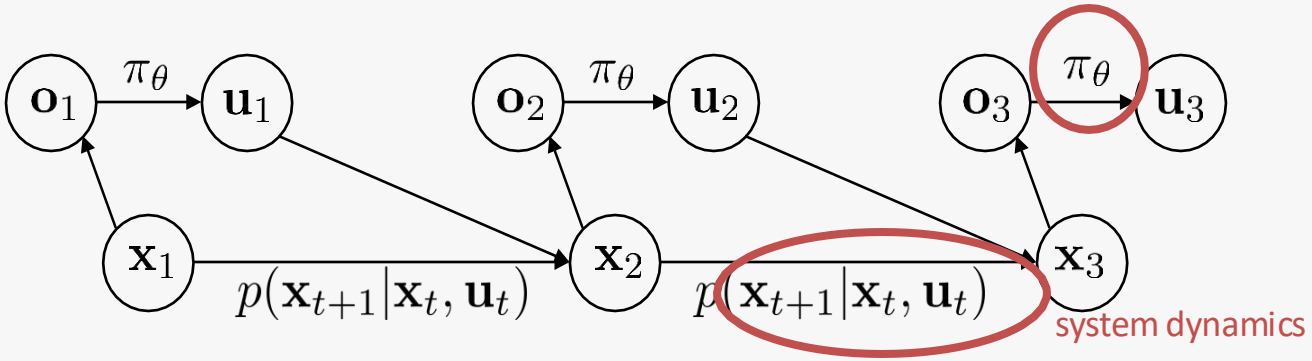
What if we knew the transition dynamics?

- Often we do know the dynamics
  1. Games (e.g., Atari games, chess, Go)
  2. Easily modeled systems (e.g., navigating a car)
  3. Simulated environments (e.g., simulated robots, video games)
- Often we can learn the dynamics
  1. System identification – fit unknown parameters of a known model
  2. Learning – fit a general-purpose model to observed transition data

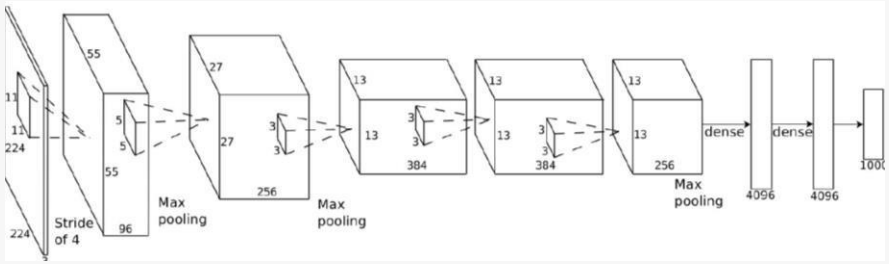
Does knowing the dynamics make things easier? Often, yes!

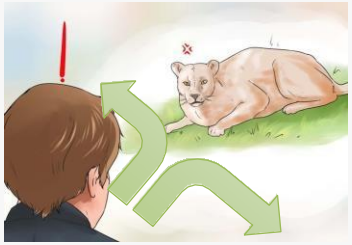
Model-based reinforcement learning

1. Model-based reinforcement learning: learn the transition dynamics, then figure out how to choose actions
2. How can we make decisions if we *know* the dynamics?
3. How can we learn *unknown* dynamics?



The objective



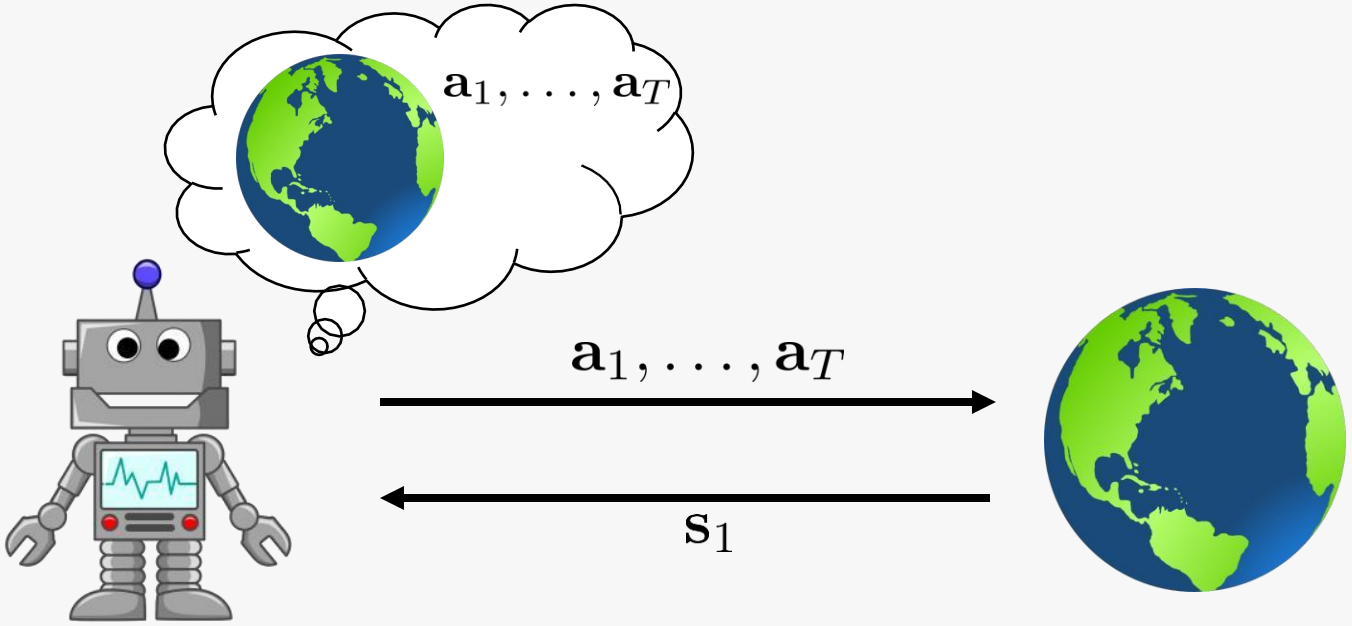


$\mathbf{s}_t$ $\pi_\theta(\mathbf{a}_t|\mathbf{s}_t)$ $\mathbf{a}_t$

$$\min_{\mathbf{a}_1,\ldots,\mathbf{a}_T} \log p(\text{eaten by tiger}|\mathbf{a}_1,\ldots,\mathbf{a}_T)$$

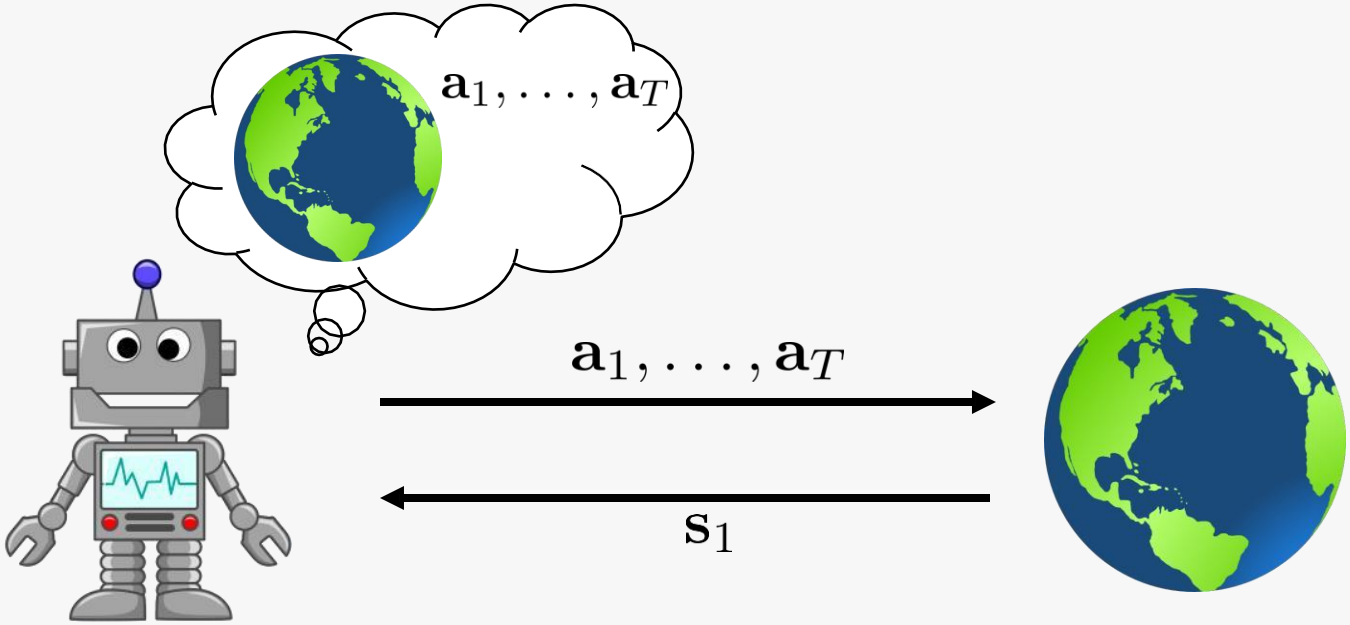
$$\min_{\mathbf{a}_1,\ldots,\mathbf{a}_T} \sum_{t=1}^{T} c(\mathbf{s}_t,\mathbf{a}_t) \text{ s.t. } \mathbf{s}_t = f(\mathbf{s}_{t-1},\mathbf{a}_{t-1})$$

The deterministic case



$$\mathbf{a}_1, \ldots, \mathbf{a}_T = \arg \max_{\mathbf{a}_1, \ldots, \mathbf{a}_T} \sum_{t=1}^{T} r(\mathbf{s}_t, \mathbf{a}_t) \text{ s.t. } \mathbf{a}_{t+1} = f(\mathbf{s}_t, \mathbf{a}_t)$$

The stochastic open-loop case



$$p_\theta(\mathbf{s}_1, \ldots, \mathbf{s}_T | \mathbf{a}_1, \ldots, \mathbf{a}_T) = p(\mathbf{s}_1) \prod_{t=1}^{T} p(\mathbf{s}_{t+1} | \mathbf{s}_t, \mathbf{a}_t)$$

$$\mathbf{a}_1, \ldots, \mathbf{a}_T = \arg \max_{\mathbf{a}_1, \ldots, \mathbf{a}_T} E \left[\sum_t r(\mathbf{s}_t, \mathbf{a}_t) | \mathbf{a}_1, \ldots, \mathbf{a}_T \right]$$

Stochastic optimization

abstract away optimal control/planning:

$$\mathbf{a}_1, \ldots, \mathbf{a}_T = \arg \max_{\mathbf{a}_1, \ldots, \mathbf{a}_T} \underbrace{J(\mathbf{a}_1, \ldots, \mathbf{a}_T)}_{\text{don't care what this is}}$$

$$\mathbf{A} = \arg \max_{\mathbf{A}} J(\mathbf{A})$$

simplest method: guess & check "random shooting method"

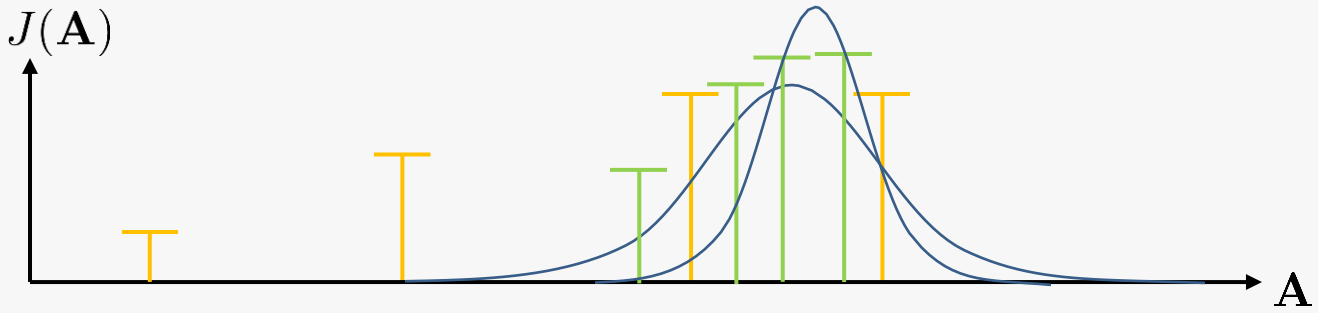
1. pick $\mathbf{A}_1, \ldots, \mathbf{A}_N$ from some distribution (e.g., uniform)
2. choose $\mathbf{A}_i$ based on $\arg\max_i J(\mathbf{A}_i)$

Cross-entropy method (CEM)

1. pick $\mathbf{A}_1, \ldots, \mathbf{A}_N$ from some distribution (e.g., uniform)
2. choose $\mathbf{A}_i$ based on $\arg\max_i J(\mathbf{A}_i)$

can we do better?

typically use Gaussian distribution



cross-entropy method with continuous-valued inputs:

1. sample $\mathbf{A}_1, \ldots, \mathbf{A}_N$ from $p(\mathbf{A})$
2. evaluate $J(\mathbf{A}_1), \ldots, J(\mathbf{A}_N)$
3. pick the *elites* $\mathbf{A}_{i_1}, \ldots, \mathbf{A}_{i_M}$ with the highest value, where $M < N$
4. refit $p(\mathbf{A})$ to the elites $\mathbf{A}_{i_1}, \ldots, \mathbf{A}_{i_M}$

What's the upside?

1. Very fast if parallelized
2. Extremely simple

What's the problem?

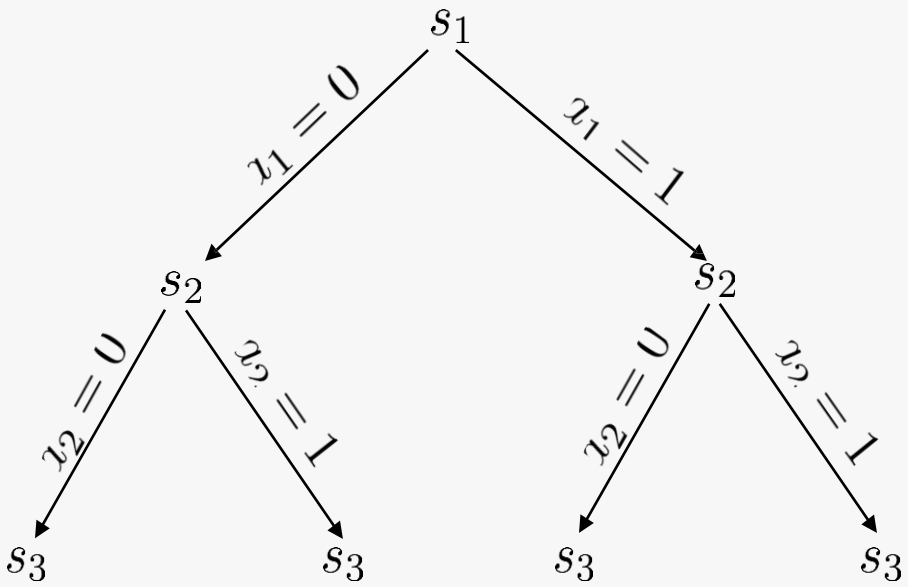
1. Very harsh dimensionality limit
2. Only open-loop planning

Discrete case: Monte Carlo tree search (MCTS)

s_t

a_t

discrete planning as a search problem



Why learn the model?

If we knew $f(\mathbf{s}_t, \mathbf{a}_t) = \mathbf{s}_{t+1}$, we could use the tools

(or $p(\mathbf{s}_{t+1}|\mathbf{s}_t, \mathbf{a}_t)$ in the stochastic case)

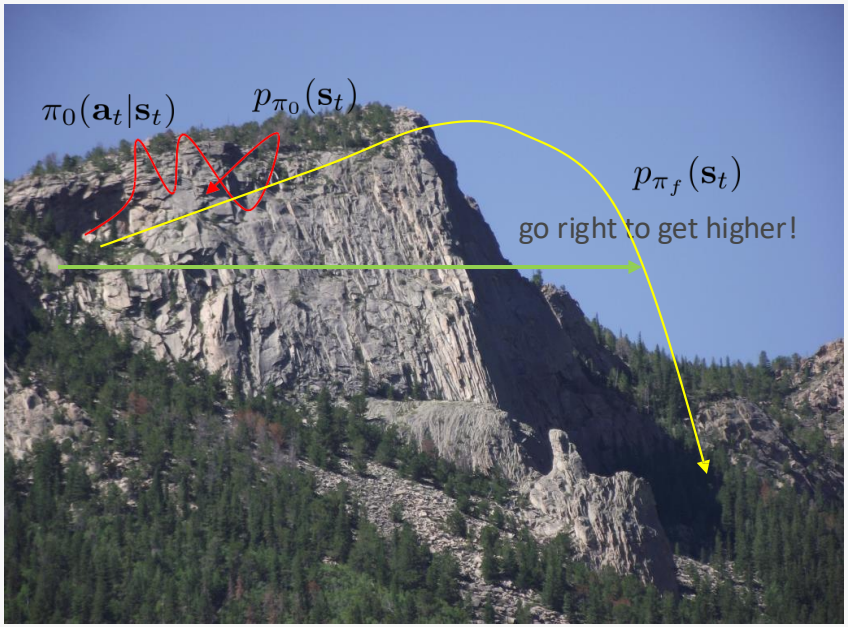
So let's learn $f(\mathbf{s}_t, \mathbf{a}_t)$ from data, and *then* plan through it!

model-based reinforcement learning version 0.5:

1. run base policy $\pi_0(\mathbf{a}_t|\mathbf{s}_t)$ (e.g., random policy) to collect $\mathcal{D} = \{(\mathbf{s}, \mathbf{a}, \mathbf{s}')_i\}$
2. learn dynamics model $f(\mathbf{s}, \mathbf{a})$ to minimize $\sum_i \|f(\mathbf{s}_i, \mathbf{a}_i) - \mathbf{s}'_i\|^2$
3. plan through $f(\mathbf{s}, \mathbf{a})$ to choose actions

Does it work? No!

1. run base policy $\pi_0(\mathbf{a}_t|\mathbf{s}_t)$ (e.g., random policy) to collect $\mathcal{D} = \{(\mathbf{s}, \mathbf{a}, \mathbf{s}')_i\}$
2. learn dynamics model $f(\mathbf{s}, \mathbf{a})$ to minimize $\sum_i \|f(\mathbf{s}_i, \mathbf{a}_i) - \mathbf{s}'_i\|^2$
3. plan through $f(\mathbf{s}, \mathbf{a})$ to choose actions



$$p_{\pi_f}(\mathbf{s}_t) \neq p_{\pi_0}(\mathbf{s}_t)$$

- Distribution mismatch problem becomes exacerbated as we use more expressive model classes

Can we do better?

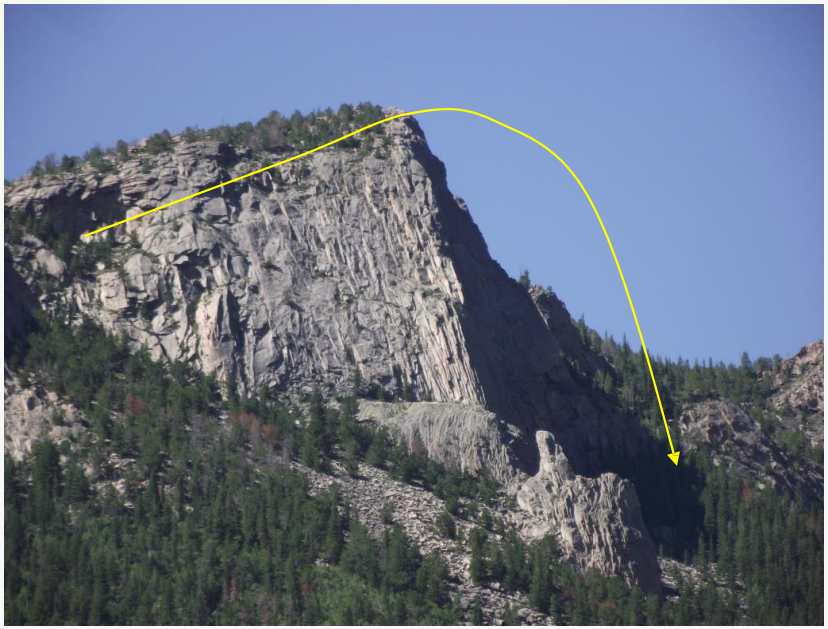
can we make $p_{\pi_0}(\mathbf{s}_t) = p_{\pi_f}(\mathbf{s}_t)$?

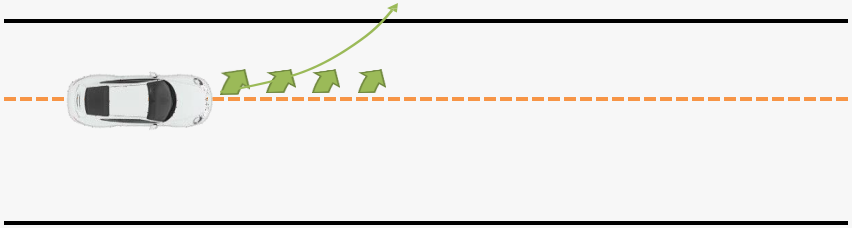
where have we seen that before? need to collect data from $p_{\pi_f}(\mathbf{s}_t)$

model-based reinforcement learning version 1.0:

1. run base policy $\pi_0(\mathbf{a}_t|\mathbf{s}_t)$ (e.g., random policy) to collect $\mathcal{D} = \{(\mathbf{s}, \mathbf{a}, \mathbf{s}')_i\}$
2. learn dynamics model $f(\mathbf{s}, \mathbf{a})$ to minimize $\sum_i \|f(\mathbf{s}_i, \mathbf{a}_i) - \mathbf{s}'_i\|^2$
3. plan through $f(\mathbf{s}, \mathbf{a})$ to choose actions
4. execute those actions and add the resulting data $\{(\mathbf{s}, \mathbf{a}, \mathbf{s}')_j\}$ to $\mathcal{D}$

What if we make a mistake?





Can we do better? --> MPC

model-based reinforcement learning version 1.5:

1. run base policy $\pi_0(\mathbf{a}_t|\mathbf{s}_t)$ (e.g., random policy) to collect $\mathcal{D} = \{(\mathbf{s}, \mathbf{a}, \mathbf{s}')_i\}$
2. learn dynamics model $f(\mathbf{s}, \mathbf{a})$ to minimize $\sum_i \|f(\mathbf{s}_i, \mathbf{a}_i) - \mathbf{s}'_i\|^2$
3. plan through $f(\mathbf{s}, \mathbf{a})$ to choose actions
4. execute the first planned action, observe resulting state $\mathbf{s}'$ (MPC)
5. append $(\mathbf{s}, \mathbf{a}, \mathbf{s}')$ to dataset $\mathcal{D}$

every N steps

How to replan? - Model Predictive Control

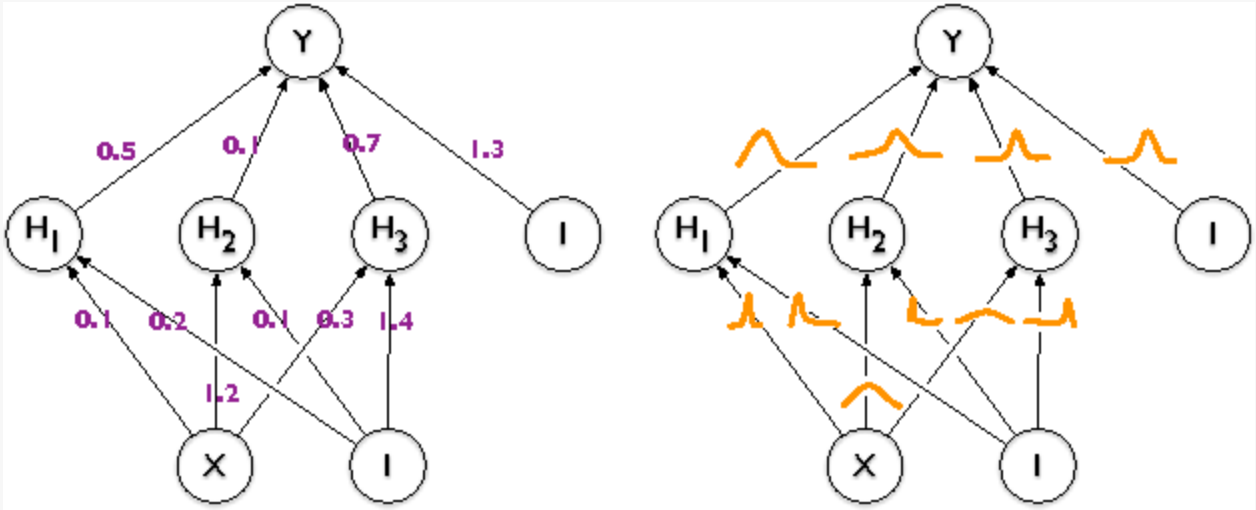
model-based reinforcement learning version 1.5:

1. run base policy $\pi_0(\mathbf{a}_t|\mathbf{s}_t)$ (e.g., random policy) to collect $\mathcal{D} = \{(\mathbf{s}, \mathbf{a}, \mathbf{s}')_i\}$
2. learn dynamics model $f(\mathbf{s}, \mathbf{a})$ to minimize $\sum_i \|f(\mathbf{s}_i, \mathbf{a}_i) - \mathbf{s}'_i\|^2$
3. plan through $f(\mathbf{s}, \mathbf{a})$ to choose actions
4. execute the first planned action, observe resulting state $\mathbf{s}'$ (MPC)
5. append $(\mathbf{s}, \mathbf{a}, \mathbf{s}')$ to dataset $\mathcal{D}$

every N steps

- The more you replan, the less perfect each individual plan needs to be
- Can use shorter horizons
- Even random sampling can often work well here!

Possible Improvements --> Bayesian neural networks



common approximation:

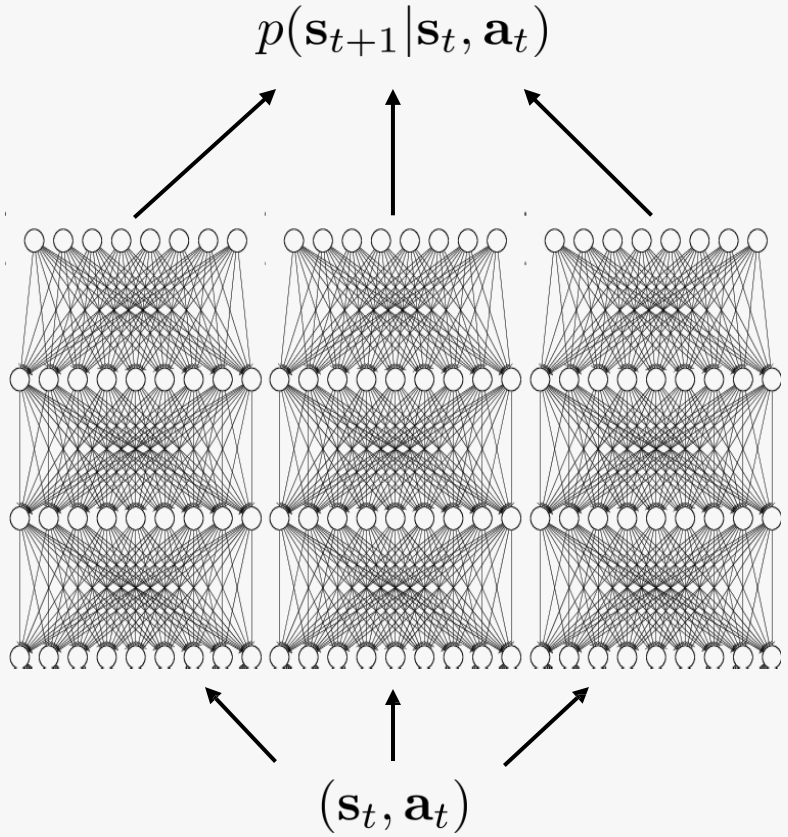
$$p(\theta|\mathcal{D}) = \prod_i p(\theta_i|\mathcal{D})$$

$$p(\theta_i|\mathcal{D}) = \mathcal{N}(\mu_i, \sigma_i)$$

expected weight

uncertainty about the weight

Possible improvements -->Bootstrap ensembles



Train multiple models and see if they agree!

formally: $p(\theta|\mathcal{D}) \approx \frac{1}{N}\sum_i \delta(\theta_i)$

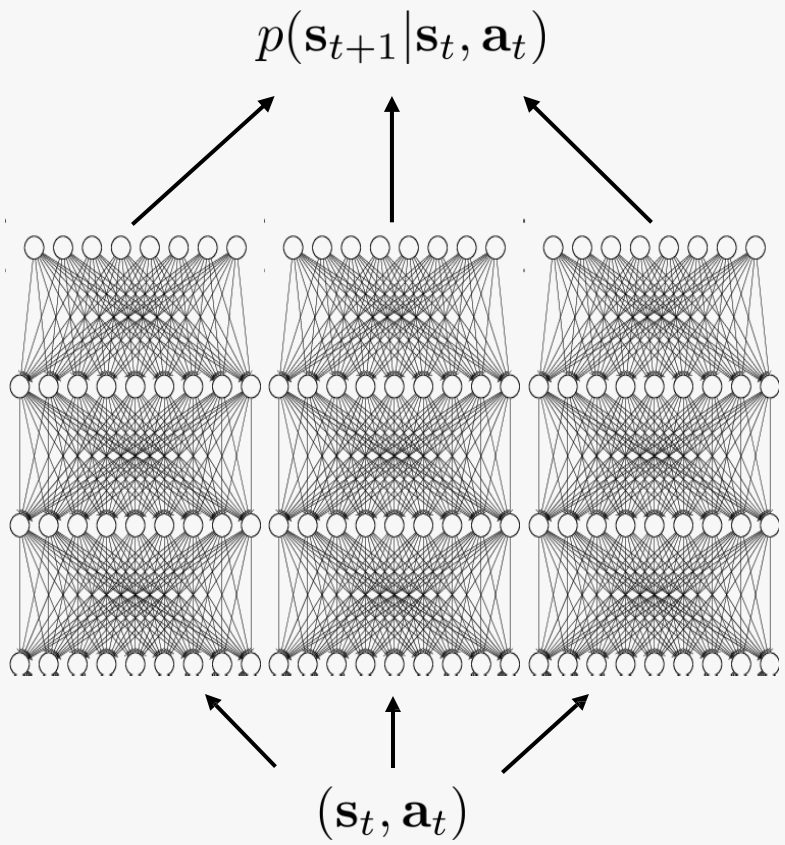
$$\int p(\mathbf{s}_{t+1}|\mathbf{s}_t, \mathbf{a}_t, \theta)p(\theta|\mathcal{D})d\theta \approx \frac{1}{N}\sum_i p(\mathbf{s}_{t+1}|\mathbf{s}_t, \mathbf{a}_t, \theta_i)$$

How to train?

Main idea: need to generate "independent" datasets to get "independent" models

θ_i is trained on $\mathcal{D}_i$, sampled *with replacement* from $\mathcal{D}$

Bootstrap ensembles in deep learning

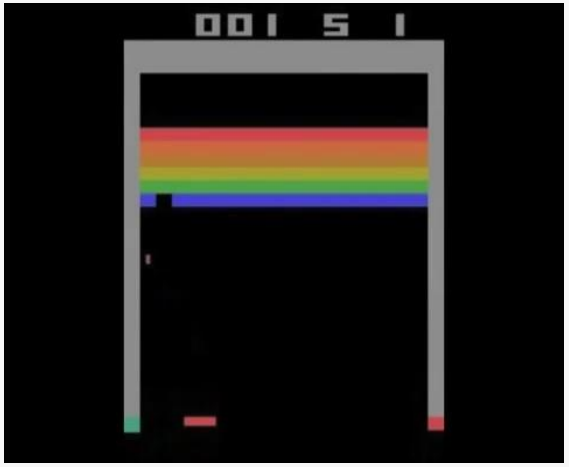


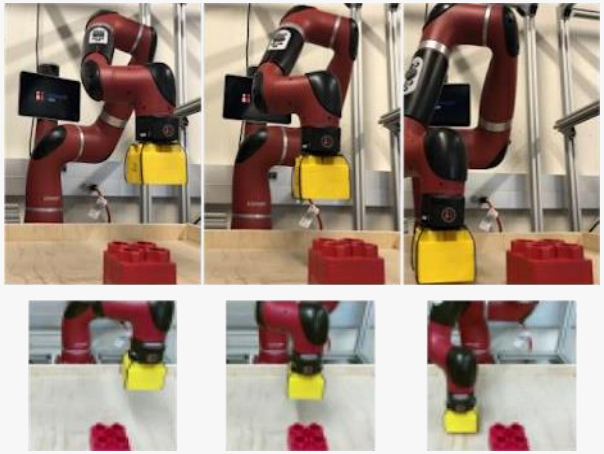
This basically works

Very crude approximation, because the number of models is usually small (< 10)

Resampling with replacement is usually unnecessary, because SGD and random initialization usually makes the models sufficiently independent

What about complex observations?

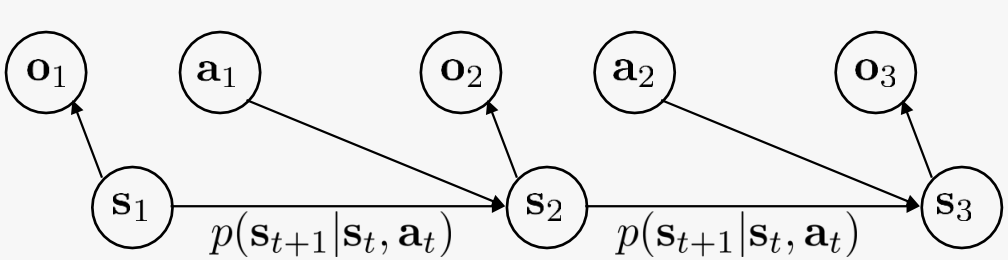




$$f(\mathbf{s}_t, \mathbf{a}_t) = \mathbf{s}_{t+1}$$

What is hard about this?

- High dimensionality
- Redundancy
- Partial observability



high-dimensional but not dynamic

low-dimension but dynamic

separately learn $p(\mathbf{o}_t|\mathbf{s}_t)$ and $p(\mathbf{s}_{t+1}|\mathbf{s}_t, \mathbf{a}_t)$?

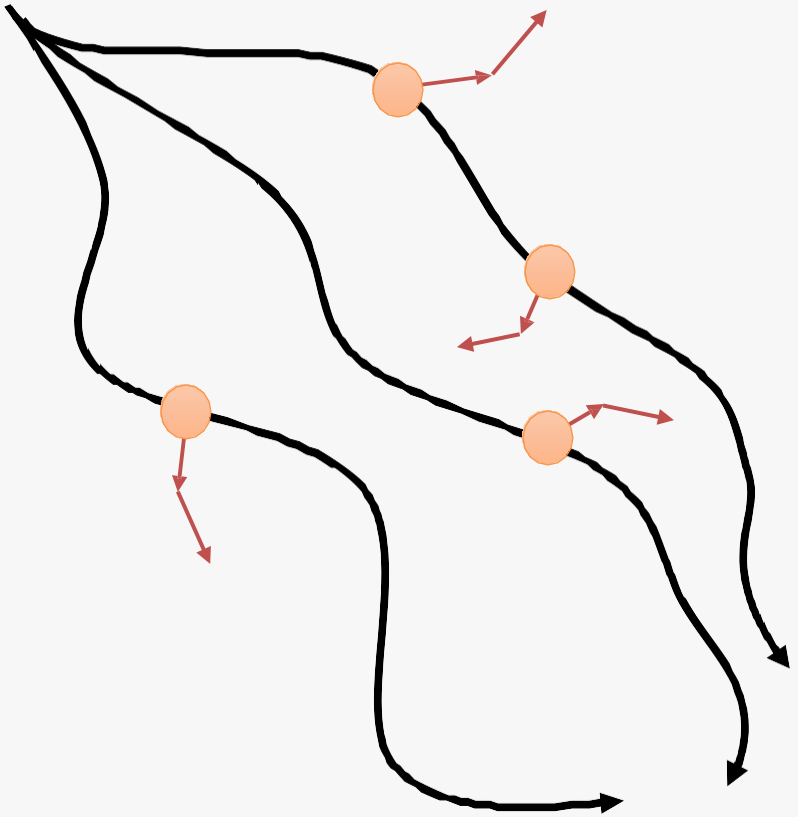
Model-Free Learning With a Model

What's the solution?

- Use derivative-free ("model-free") RL algorithms, with the model used to generate synthetic samples
  - Seems weirdly backwards
  - Actually works very well
  - Essentially "model-based acceleration" for model-free RL

General "Dyna-style" model-based RL recipe

1. collect some data, consisting of transitions (s, a, s', r)
2. learn model $\hat{p}(s'|s, a)$ (and optionally, $\hat{r}(s, a)$)
3. repeat K times:
   4. sample $s \sim \mathcal{B}$ from buffer
   5. choose action a (from $\mathcal{B}$, from π, or random)
   6. simulate $s' \sim \hat{p}(s'|s, a)$ (and $r = \hat{r}(s, a)$)
   7. train on (s, a, s', r) with model-free RL
   8. (optional) take N more model-based steps

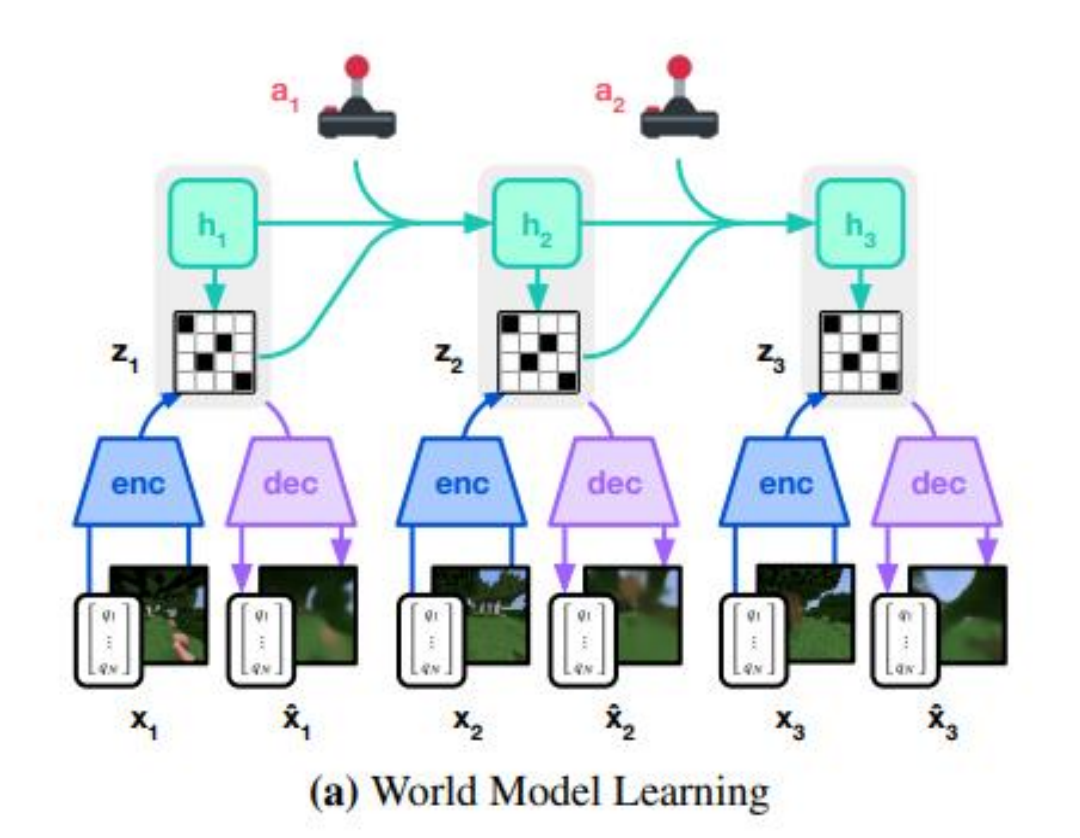


+ only requires short (as few as one step) rollouts from model

+ still sees diverse states

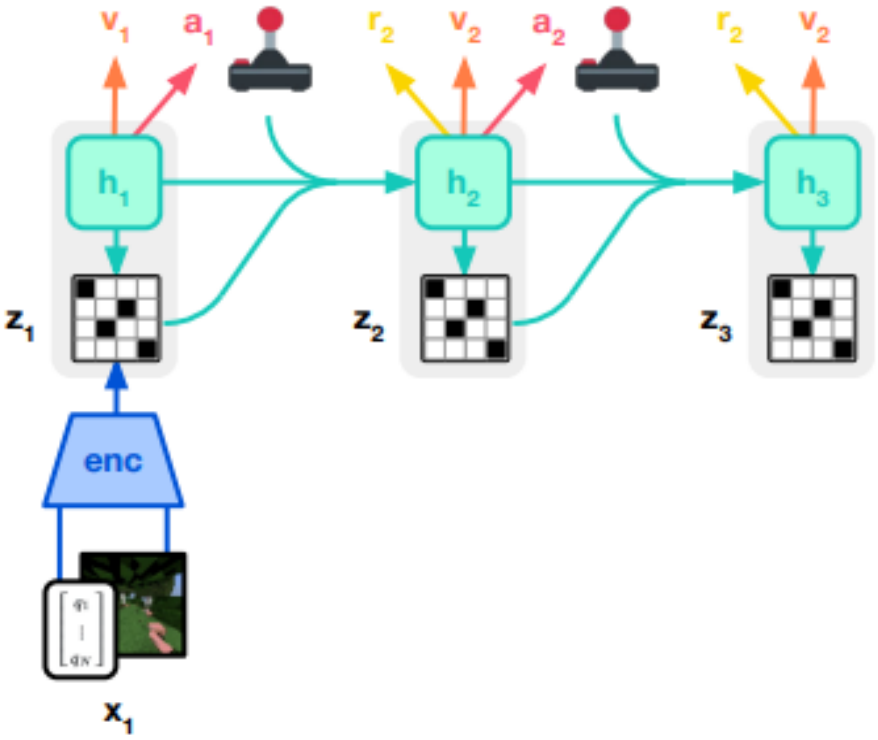
Imagination Based MBRL

Dreamer Algorithm



(a) World Model Learning

Dreamer Algorithm



(b) Actor Critic Learning

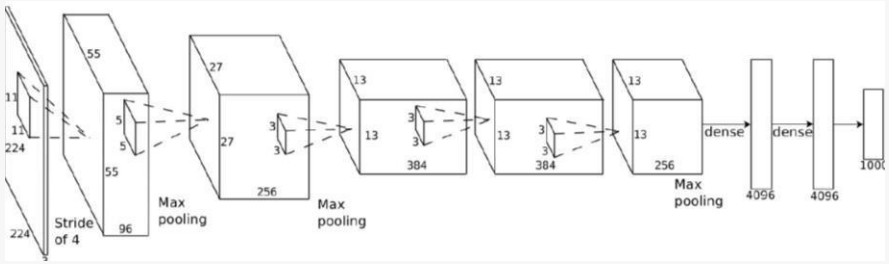
Advanced World Models

- **Generative World Models:** Train models to predict observations directly. Simple (e.g., Transformer Dynamics Model), high-dimensional (e.g., diffusion models).
- **Generative Models with Latent Variables:** Use latent variables to improve efficiency and reduce dimensionality. DreamerV3 is a prime example.
- **Non-Generative World Models:** Focus on other prediction targets, including values, rewards, and latent states.
- **Partial Observation Prediction:** Focus on predicting essential aspects of the observations.
- **Multi-step World Models:** Predicting several time steps ahead.

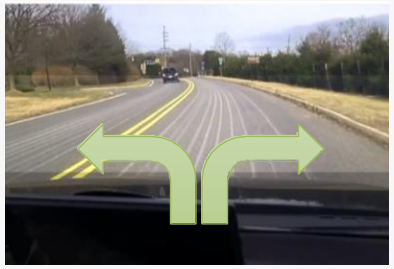
Imitation Learning / Offline RL

Imitation Learning

$\mathbf{o}_t$



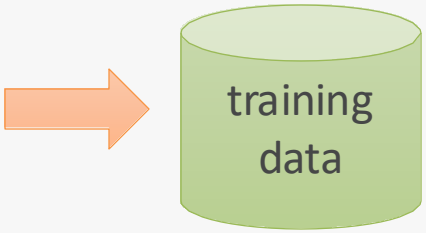
$\pi_\theta(\mathbf{a}_t|\mathbf{o}_t)$

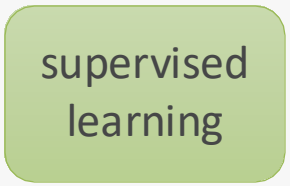


$\mathbf{a}_t$

$\mathbf{o}_t$

$\mathbf{a}_t$



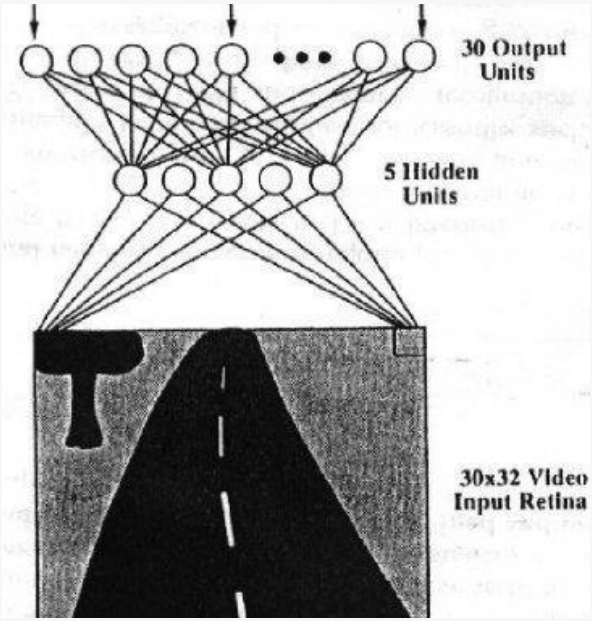


$\pi_\theta(\mathbf{a}_t|\mathbf{o}_t)$

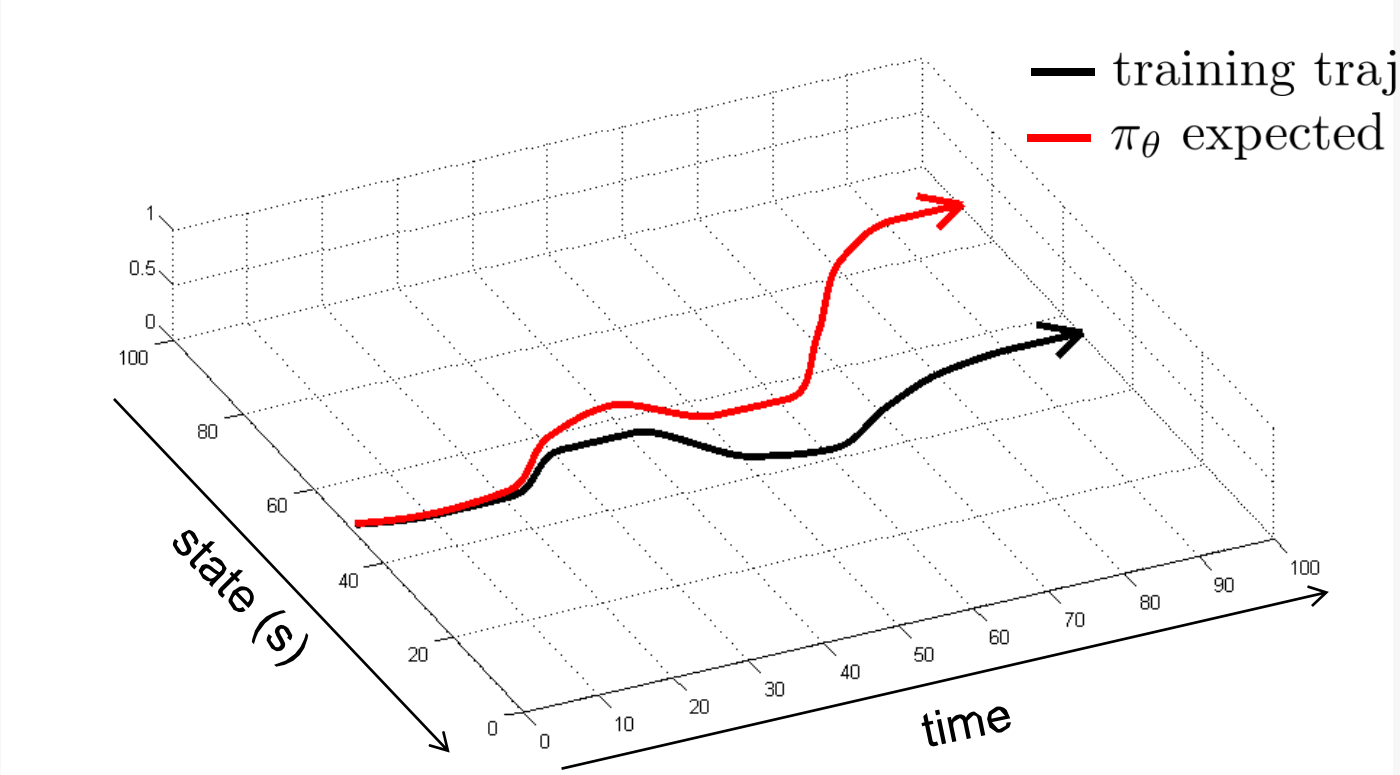
behavioral cloning

The original deep imitation learning system

ALVINN: **A**utonomous **L**and **V**ehicle **I**n a **N**eural **N**etwork 1989



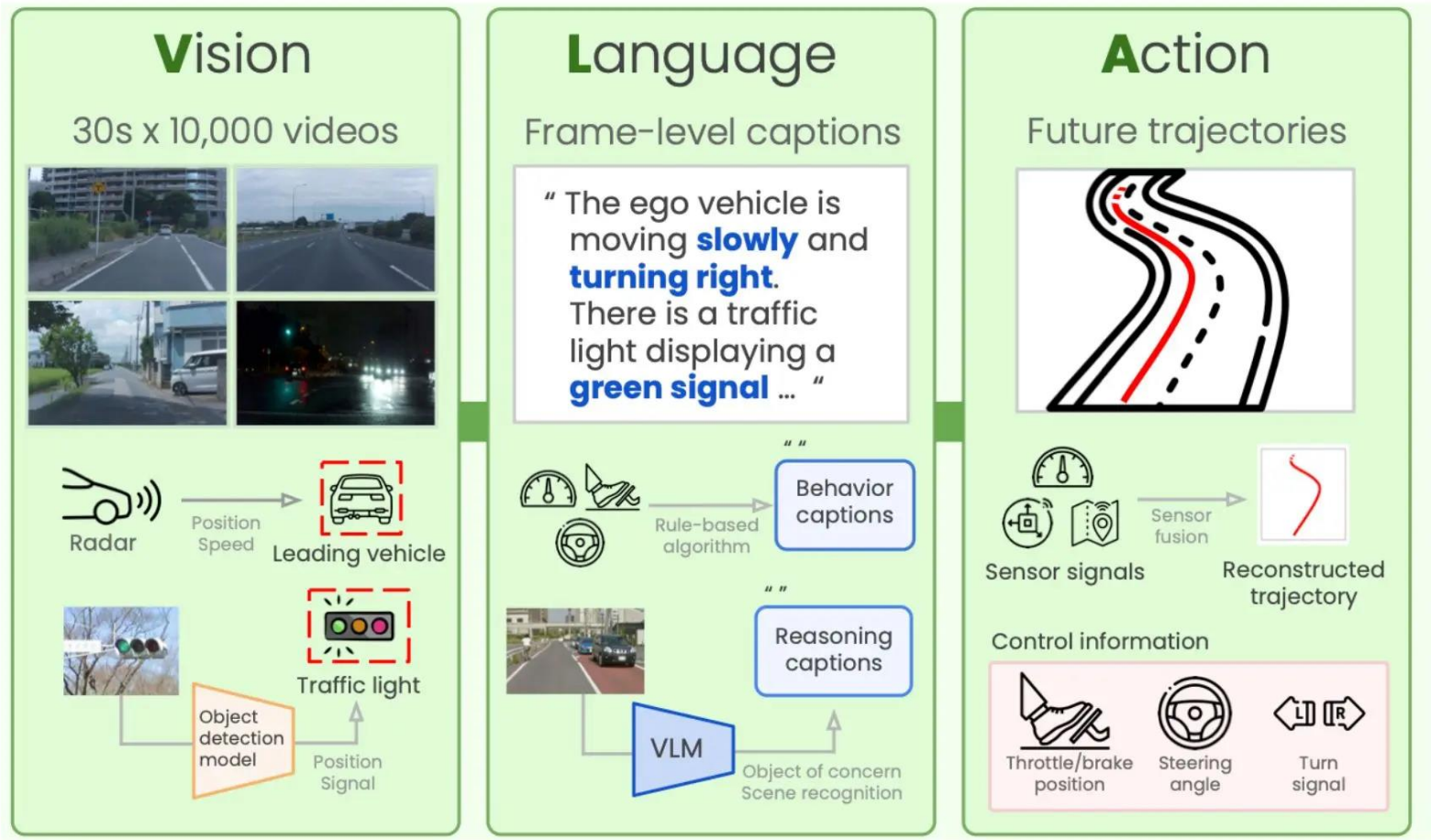
Does it work? No!



Current solutions?

- Large Generative Modeling – Stochastic:
  - Diffusion Policies
  - Visual Language Action Models (Transformers, Diffusion Models)

Visual-Language-Action Models

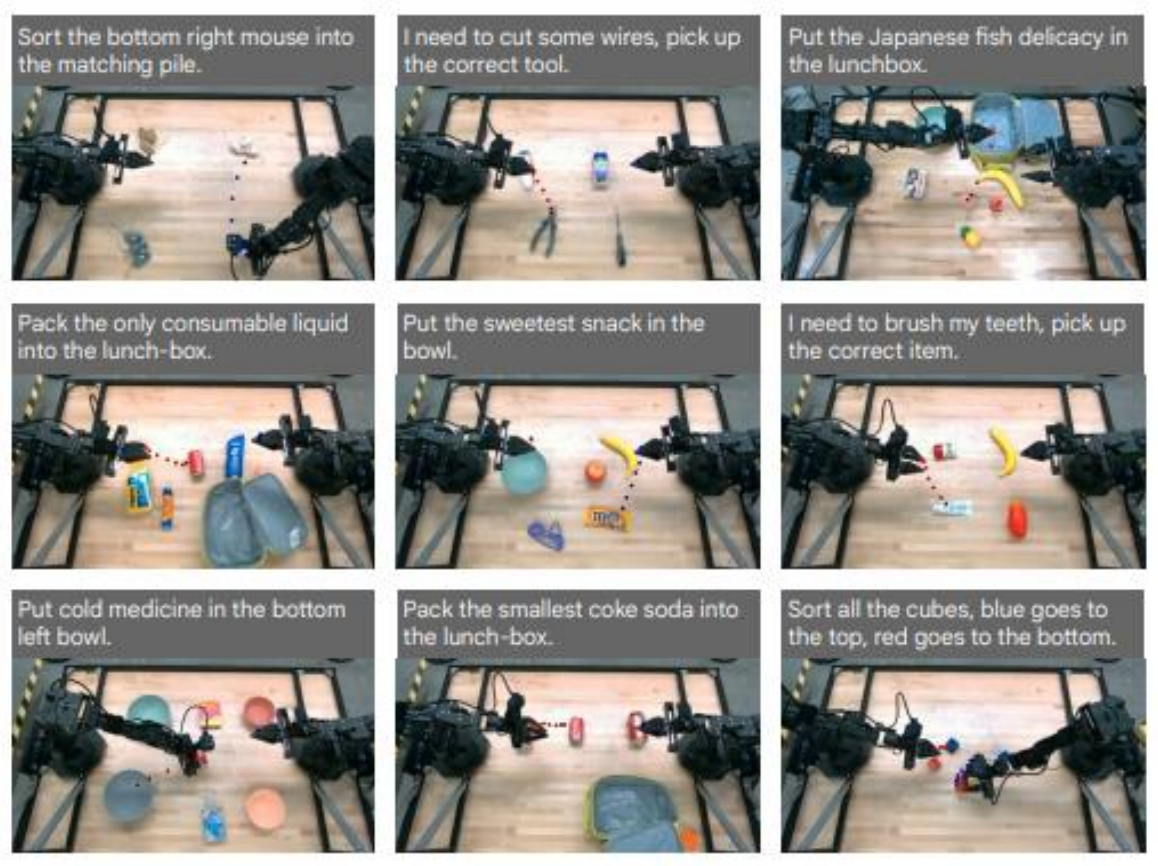


Visual-Language-Action Models

VLA models are *all-in-one AI systems* that merge:

1. **Vision**: What the robot sees (using cameras/sensors).
2. **Language**: What the robot is told (via voice or text commands).
3. **Action**: What the robot does (movement or digital tasks).

Visual-Language-Action Models



https://www.youtube.com/watch?v=4MvGnmmP3c0

Possible Aerospace applications of VLAs??

- UAVs?
- Space exploration?
- Something else?

Offline Reinforcement Learning

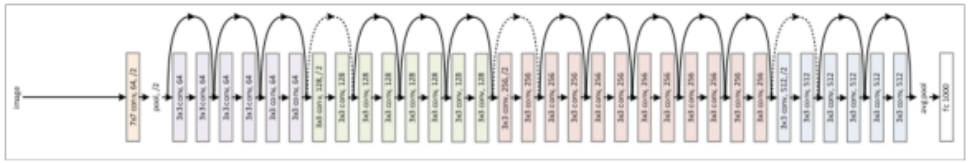
Can we apply standard RL in the real-world?

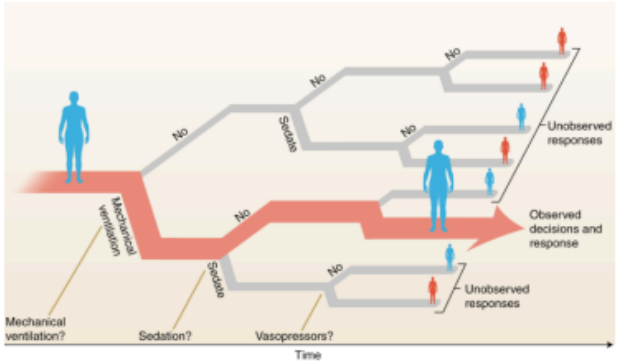
- RL is fundamentally an “active” learning paradigm: the agent needs to collect its own dataset to learn meaningful policies

- This can be unsafe or expensive in real world problems!

Generalization?

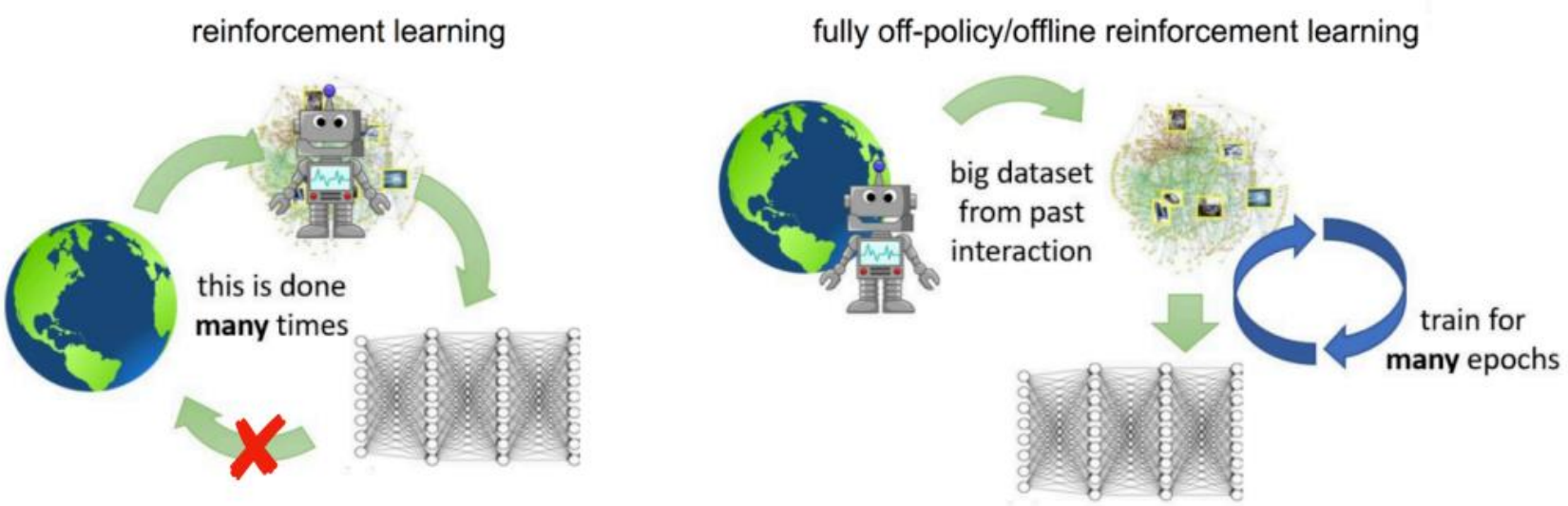
?





Iterated data collection can cause poor generalization!

Offline (Batch) Reinforcement Learning



Learn from a previously collected static dataset

Why is offline RL promising?

- Large static datasets of meaningful behaviours already exist
- Large datasets at the core of successes in Vision and NLP

How good can offline RL perform?

Supervised Learning

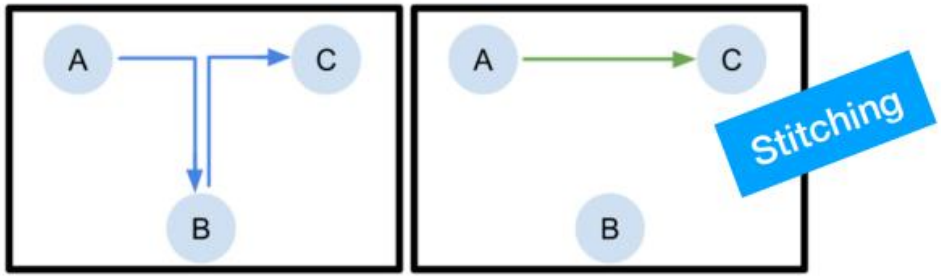
→ Dog

Can do as good as the dataset!

→ Cat?

Offline Reinforcement Learning

Can do **better** than the dataset!



Can show that Q-learning recovers optimal policy from random data.

RL for LLMS

Role of RL in Large Language Models (LLMs)

We want our LLMs to be more than just good at stringing words together. We want them to be:

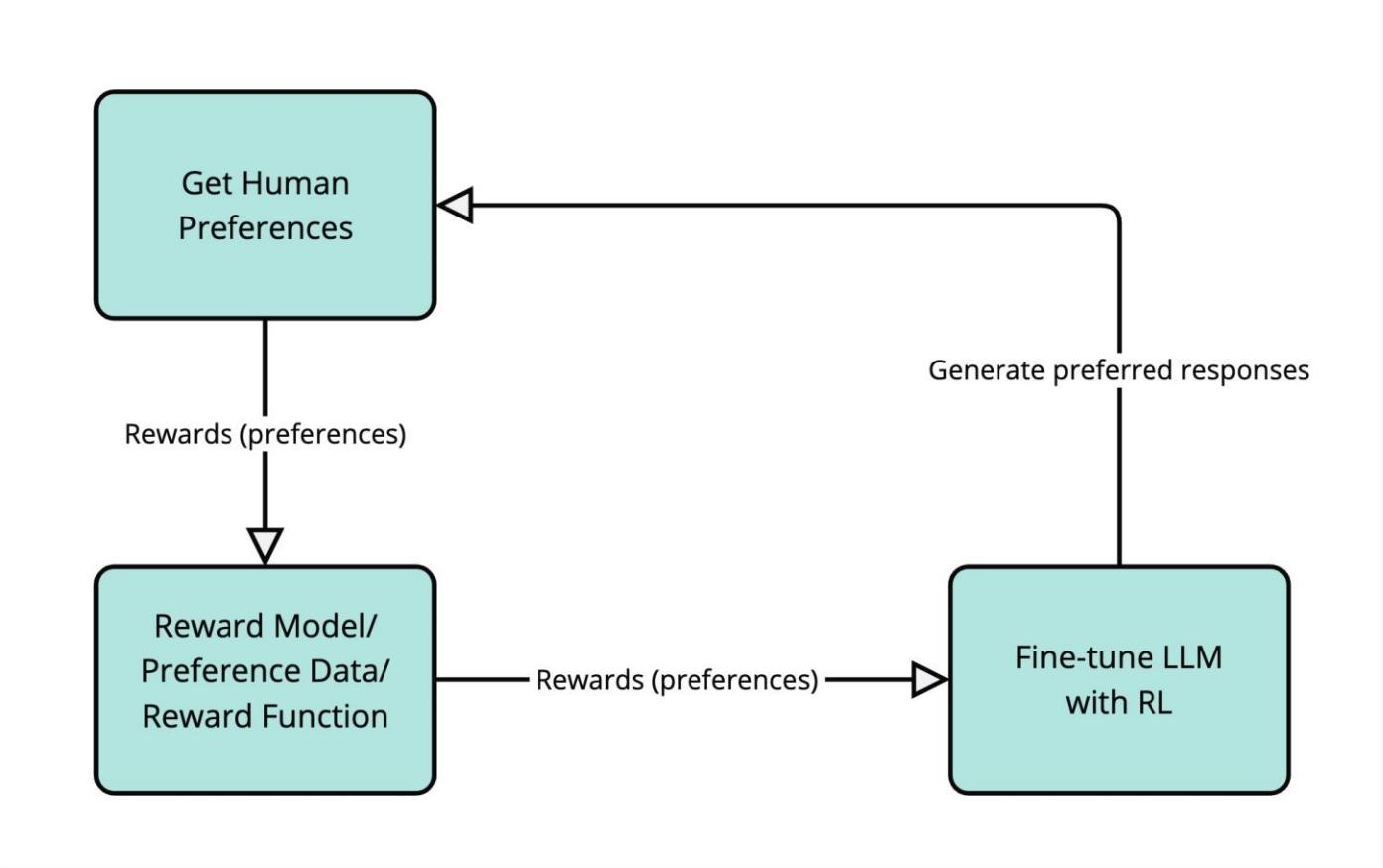
- **Helpful:** Provide useful and relevant information.
- **Harmless:** Avoid generating toxic, biased, or harmful content.
- **Aligned with Human Preferences:** Respond in ways that humans find natural, helpful, and engaging.

Reinforcement Learning from Human Feedback (RLHF)

A very popular technique for aligning language models is **Reinforcement Learning from Human Feedback (RLHF)**. In RLHF, we use human feedback as a proxy for the "reward" signal in RL. Here's how it works:

- **Get Human Preferences:** We might ask humans to compare different responses generated by the LLM for the same input prompt and tell us which response they prefer. For example, we might show a human two different answers to the question "What is the capital of France?" and ask them "Which answer is better?".
- **Train a Reward Model:** We use this human preference data to train a separate model called a **reward model**. This reward model learns to predict what kind of responses humans will prefer. It learns to score responses based on helpfulness, harmlessness, and alignment with human preferences.
- **Fine-tune the LLM with RL:** Now we use the reward model as the environment for our LLM agent. The LLM generates responses (actions), and the reward model scores these responses (provides rewards). In essence, we're training the LLM to produce text that our reward model (which learned from human preferences) thinks is good.

Reinforcement Learning from Human Feedback (RLHF)



LLM reasoning: Chain of thought

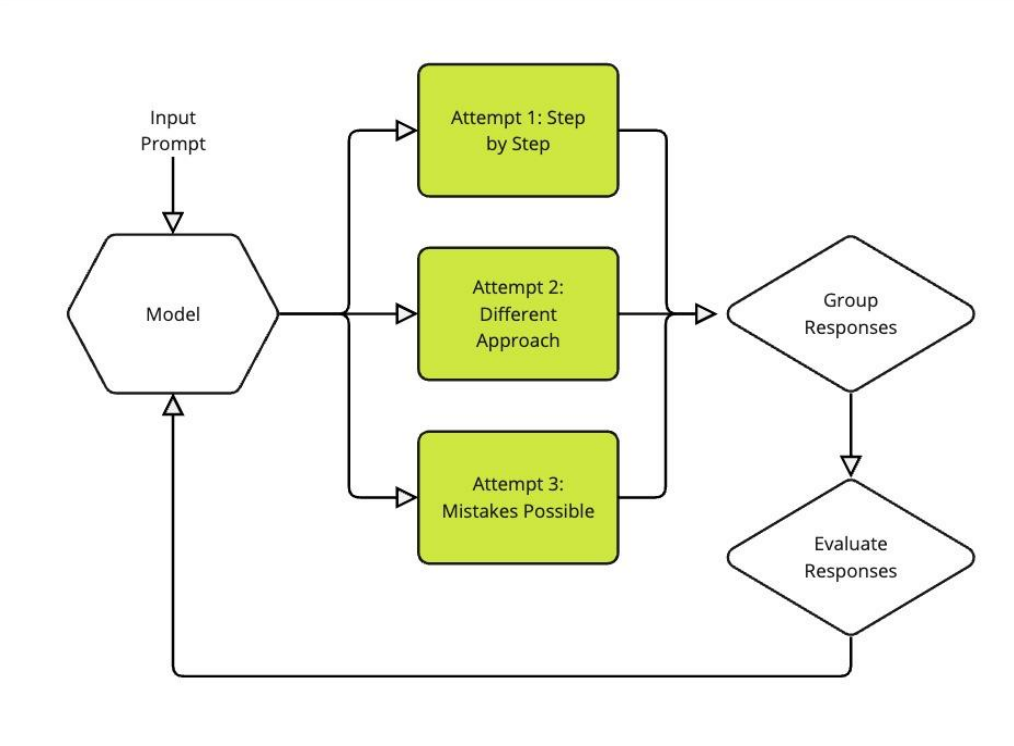
```
Problem: "I have 3 apples and 2 oranges. How many pieces of fruit do I have in total?"

Thought: "I need to add the number of apples and oranges to get the total number of pieces of fruit."

Answer: "5"
```

DeepSeek's innovation: Group Relative Policy Optimization (GRPO)

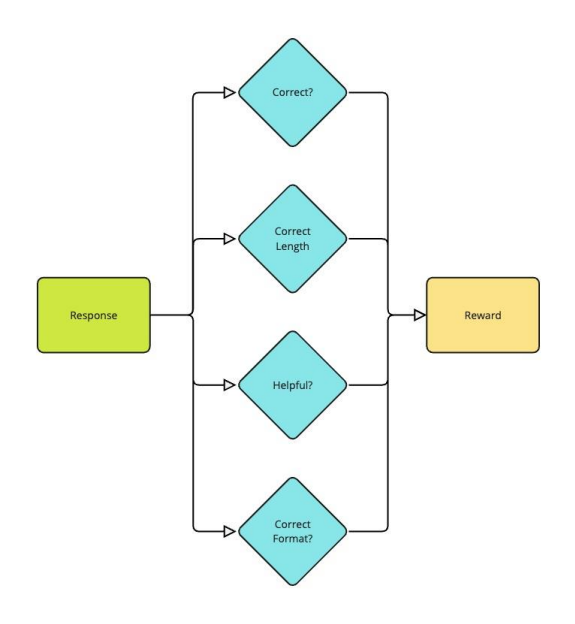
A variant of PPO, but simpler... It avoids the value function by grouping possible responses



DeepSeek's innovation: Group Relative Policy Optimization (GRPO)

The evaluation process looks at various aspects of each solution:

- Is the final answer correct?
- Did the solution follow proper formatting (like using the right XML tags)?
- Does the reasoning match the answer provided?



Acknowlegments

- CS 285 at UC Berkeley, Deep Reinforcement Learning. Sergey Levine
- Reinforcement Learning: A Comprehensive Overview. Kevin P. Murphy
- Advanced Topics in Learning and Decision Making, Pieter Abbeel. UC Berkeley
- HuggingFace LLM course